COURSE PRJECT REPORT
ON

SMART BOARD HAND GESTURE CONTROL SYSTEM USING ARDUINO AND ULTRASONIC SENSOR

By

1.Srajan Gupta - 24BEC0066

2.Harshitha - 23BEC0392

3.S.Subhashree - 24BEC0659

4. Aiswarya.S - 24BEC0671

SENSORS TECHNOLOGY[BECE409E]

FALL SEMESTER 2025

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

TABLE OF CONTENTS

1. INTRODUCTION

Human-machine interaction (HMI) has experienced a paradigm shift over the last decade. The evolution from conventional interfaces such as mechanical switches, keyboards, and touchscreen panels to more intuitive interaction methods like voice commands and gesture recognition marks a significant milestone in technology. This shift has been driven by the growing need for more natural, hands-free, and hygienic modes of interaction with electronic systems, especially in environments such as healthcare facilities, smart homes, industrial automation, and public installations where traditional interfaces may not be ideal.

Gesture-based control systems offer a compelling solution to these modern interaction challenges. These systems allow users to convey commands to machines through hand movements or body gestures, eliminating the need for physical contact. The advantage of such systems becomes particularly evident during situations where hygiene and accessibility are critical. For example, in the wake of global health concerns, the demand for **contactless interfaces** has surged dramatically, fueling the exploration and development of more intuitive gesture control mechanisms.

The primary objective of this project is to design and develop a gesture-controlled interface using low-cost, easily accessible electronic components. The project utilizes **two HC-SR04 ultrasonic distance sensors**, interfaced with an **Arduino UNO microcontroller**. These components form the core of a system capable of recognizing **Volume UP/DOWN, FORWARD, REWIND, PLAY/PAUSE, LEFT, RIGHT,** and **CLICK** gestures. Upon detecting these gestures, Arduino transmits the command to a graphical user interface built using the Processing IDE which is then coded and executed into play with the help of Python programming, which enhances the system's interactivity.

This system is designed to strike a balance between functionality and simplicity, aiming to provide a foundational approach to gesture control that is suitable for:

- **Hobbyist projects**
- **Smart home automation**
- **Touchless control panels**
- **Assistive technologies for users with limited mobility**

In addition to its practical applications, this project serves as a valuable educational platform. It introduces students and enthusiasts to key concepts in embedded systems, sensor interfacing, serial communication, and software-hardware integration. Unlike high-end gesture systems that rely on advanced image processing or machine learning techniques, this system focuses on distance-based gesture detection, which is relatively simple to implement and requires fewer computational resources. In conclusion, this project demonstrates how gesture-based interaction can be implemented using minimal hardware and basic coding principles. While it serves as a fundamental model, it holds vast potential for

expansion. By integrating additional sensors, wireless modules, or AI-based recognition in the future, this system can be evolved into a full-fledged smart gesture recognition platform for more sophisticated applications.

1.1. Literature Survey

Introduction to Gesture Recognition

Gesture recognition refers to the capability of a system to identify and interpret human motions—typically hand or body movements—as input commands. This area of study falls under the broader domain of Human-Computer Interaction (HCI) and is considered an enabling technology for developing touchless interfaces. The goal is to make communication between humans and machines more intuitive, much like natural human-to-human interaction. Various methods have been developed over the years to facilitate gesture recognition, ranging from image processing techniques using cameras to proximity sensing using ultrasonic or infrared sensors. Historically, gesture recognition systems have been extensively explored in academic research, industrial automation, gaming, and assistive technologies. The literature reveals a wide spectrum of approaches, each with different strengths, weaknesses, and hardware/software requirements.

Sensor-Based Gesture Recognition Systems

Sensor-based systems offer an affordable and resource-efficient alternative to vision-based solutions. These systems make use of devices like **accelerometers, ultrasonic sensors, IR sensors, and RFID** modules to detect motion, distance, or proximity. The sensor readings are then analysed using microcontrollers or microprocessors to interpret specific gestures.

Accelerometer-Based Systems: Accelerometers have been widely used for gesture recognition in wearable devices like smartwatches and fitness trackers. The MPU6050 (a 6-axis IMU sensor) is a popular choice for such applications.

• In 2014, M. Asci et al. proposed a gesture-based mouse control using a glove equipped with an accelerometer. While accurate, it required the user to wear the device, reducing user comfort.

• Piyush et al. (2016) developed a gesture-based robot controller using an ADXL345 accelerometer module and Arduino, demonstrating low-latency control in multiple directions.

Ultrasonic Sensor-Based Systems: The HC-SR04 ultrasonic sensor is widely used due to its ability to measure distance accurately within 2–400 cm. Some studies explored using dual or Multi ultrasonic sensor setups to detect hand motion over an axis.

• Ravi Kishore and colleagues (2018) demonstrated a simple gesture-based switch control using a single ultrasonic sensor to detect gestures in forward and backward directions.

• Bansal et al. (2020) used three ultrasonic sensors placed in an arc to recognize 2D hand movements and control lighting systems accordingly.

IR Sensor-Based Systems: Infrared sensors, especially obstacle detection modules, have been used for close-proximity sensing, particularly for detecting the presence of an object (hand) or a "tap" gesture.

• Manoj Kumar et al. (2019) designed a simple contactless switch using an IR proximity sensor to toggle devices with a single hand wave.

• In 2021, Kavya and team integrated both IR and ultrasonic sensors in a hybrid gesture control system for smart homes.

Gesture Interfaces Using Arduino and Processing

The Arduino platform, due to its affordability and open-source nature, has been extensively adopted in gesture-based HCI research and prototyping. When paired with the Processing IDE, which provides graphical outputs and visualization tools, it becomes a powerful tool for rapid development. • In 2015, Supriyo and Rajat developed a gesture-based presentation controller using ultrasonic sensors and Arduino. Their Processing sketch interpreted gestures as next/previous slide commands. • In 2018, a team from the University of Delhi presented a gesture-based volume controller where Processing generated an on-screen volume knob and gestures were mapped to increase/decrease actions. The integration of Arduino and Processing offers real-time feedback, GUI development, and communication via the serial port—exactly what this project leverages. The current system builds on such prior work by adding layered feedback (LCD and GUI), directional detection, and click recognition.

1.2. Aim

To make Smart Board Hand gesture control system using Arduino and Ultrasonic sensors.

1.3. Objectives

To enable PLAY, PAUSE, FORWARD, AND REWIND controls in a media player.

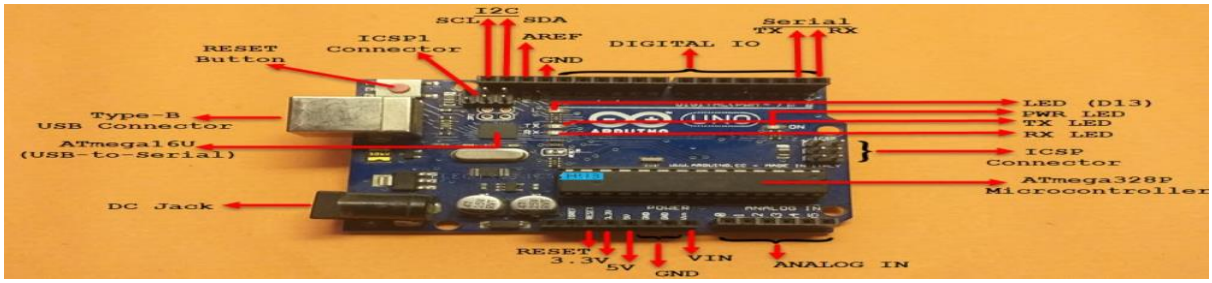
2. HARDWARE COMPONENTS USED

Arduino Uno

The Arduino UNO is considered as the powerful board used in various projects. Arduino UNO is based on an ATmega328P microcontroller.

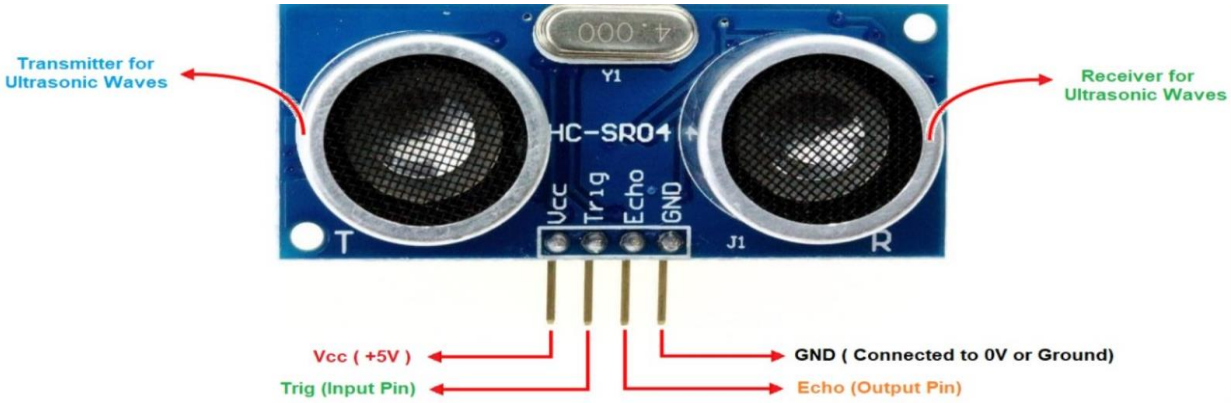
It is easy to use compared to other boards, such as the Arduino Mega board, etc. The board consists of digital and analog Input/Output pins (I/O), shields, and other circuits.

The Arduino UNO includes 6 analog pin inputs, 14 digital pins, a USB connector, a power jack, and an ICSP (In-Circuit Serial Programming) header. It is programmed based on IDE, which stands for Integrated Development Environment. It can run on both online and offline platforms.



HC-SR04

Ultrasonic sensors works on a principle similar to radar or sonar which evaluates attributes of a target by interpreting the echoes from radio or sound waves respectively. Ultrasonic sensors generate high frequency sound waves and evaluate the echo which is received back by the sensor.



Jumper Wires

3. WORK DONE

DESIGN OF THIS PROJECT

The design of the circuit is very simple, but the setup of the components is very important. The Trigger and Echo Pins of the first Ultrasonic Sensor (that is placed on the left of the screen) are connected to Pins 5 and 6of the Arduino. For the second Ultrasonic Sensor, the Trigger and Echo Pins are connected to Pins 9 and 10 of the Arduino.

Now, coming to the placement of the Sensors, place both the Ultrasonic Sensors on top of the Laptop screen, one at the left end and the other at right. You can use double sided tape to hold the sensors onto the screen.

Coming to Arduino, place it on the back of the laptop screen. Connect the wires from Arduino to Trigger and Echo Pins of the individual sensors.

Now, we are ready for programming the Arduino.

Tasks Performed:

- Switch to Next Tab in a Web Browser
- Switch to Previous Tab in a Web Browser
- Scroll Down in a Web Page
- Scroll Up in a Web Page
- Increase Volume
- Decrease Volume

Hand Gestures:

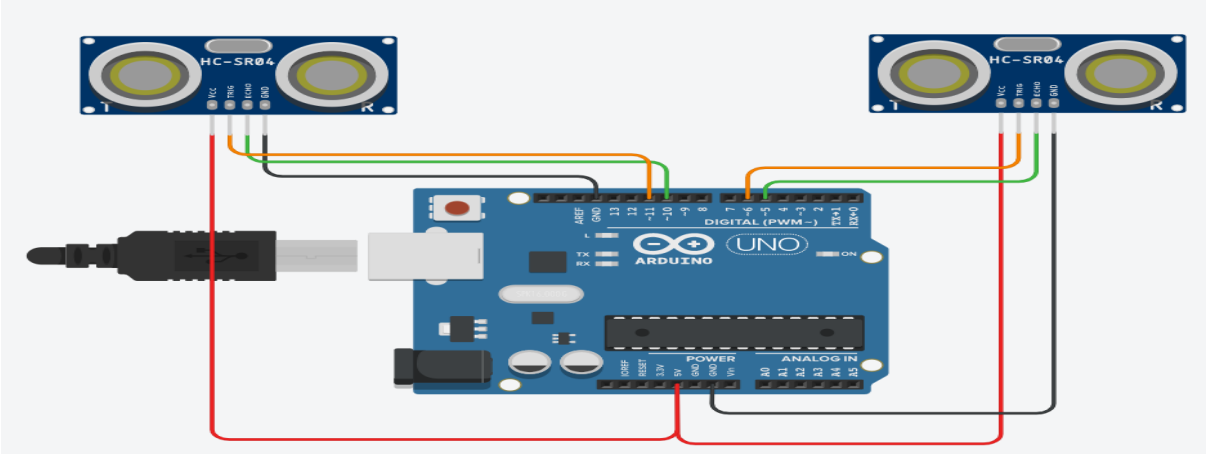
Play / Pause Gesture: When both hands are placed at a distance between 25 cm and 50 cm from the two ultrasonic sensors, the system detects this as a Play/Pause command. This gesture indicates that both hands are in the mid-range zone simultaneously, which the system interprets as a signal to toggle between play and pause functions, providing a simple and contactless way to control media playback.

Left Hand – Volume Control Gesture: Holding the left hand between 10 cm and 20 cm from the left ultrasonic sensor activates the Volume Control Mode. Once this mode is active, moving the left hand closer to the sensor (less than 15 cm) increases the volume (Volume Up), while pulling the hand slightly away (more than 20 cm but less than 40 cm) decreases the volume (Volume Down). This allows smooth, gesture-based volume adjustment.

Right Hand – Media Control Gesture: When the right hand is held at a distance between 10 cm and 20 cm from the right ultrasonic sensor, the system switches to the Media Control Mode. In this mode, pushing the right hand closer to the sensor (below 15 cm) performs a Rewind action, whereas pulling the hand slightly away (above 20 cm but under 40 cm) performs a Forward or next-section command, allowing intuitive track navigation.

Swipe Next Gesture: A quick movement of the right hand very close to the sensor (within 8 cm for around 300 milliseconds) is detected as a Swipe Next gesture. This gesture is designed for fast control, enabling the user to quickly skip to the next track or action with a simple hand wave near the right sensor.

Circuit Diagram:



4. SOFTWARE CODING

Code: (Arduino code)

```
const int trigger1 = 5;
const int echo1 = 6;
const int trigger2 = 9;
const int echo2 = 10;

  long time_taken;

int dist, distL, distR;

 long duration;

float r;
unsigned long temp = 0;
int temp1 = 0;
int l = 0;

void find_distance(void);
void calculate_distance(int trigger, int echo);

void find_distance(void) {
   digitalWrite(trigger1, LOW);
   delayMicroseconds(2);
   digitalWrite(trigger1, HIGH);
   delayMicroseconds(10);
   digitalWrite(trigger1, LOW);
   duration = pulseIn(echo1, HIGH, 5000);
   r = 3.4 * duration / 2;
   distL = r / 100.00;

   digitalWrite(trigger2, LOW);
   delayMicroseconds(2);
   digitalWrite(trigger2, HIGH);
   delayMicroseconds(10);
   digitalWrite(trigger2, LOW);
   duration = pulseIn(echo2, HIGH, 5000);
   r = 3.4 * duration / 2;
   distR = r / 100.00;
   delay(100);
}
```

```
void setup() {
  Serial.begin(9600);
  pinMode(trigger1, OUTPUT);
  pinMode(echo1, INPUT);
  pinMode(trigger2, OUTPUT);
  pinMode(echo2, INPUT);
}

void calculate_distance(int trigger, int echo) {
  digitalWrite(trigger, LOW);
  delayMicroseconds(2);
  digitalWrite(trigger, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigger, LOW);
  time_taken = pulseIn(echo, HIGH);
  dist = time_taken * 0.034 / 2;
  if (dist > 50)
    dist = 50;
}

void loop() {
  calculate_distance(trigger1, echo1);
  distL = dist;

  calculate_distance(trigger2, echo2);
  distR = dist;

  Serial.print("L=");
  Serial.println(distL);
  Serial.print("R=");
  Serial.println(distR);


  if ((distL > 25 && distR > 25) && (distL < 50 && distR < 50)) {
    Serial.println("Play/Pause");
    delay(500);
  }

  calculate_distance(trigger1, echo1);
  distL = dist;

  calculate_distance(trigger2, echo2);
  distR = dist;
  if (distL >= 10 && distL <= 20) {
    delay(50);
    calculate_distance(trigger1, echo1);
    distL = dist;
    if (distL >= 10 && distL <= 20) {
      Serial.println("Left Locked");
      while (distL <= 40) {
        calculate_distance(trigger1, echo1);
        distL = dist;
        if (distL < 15) {
          Serial.println("Vup");
          delay(300);
```

```
            }
            if (distL > 20) {
               Serial.println("Vdown");
               delay(300);
            }
         }
      }
   }

   if (distR >= 10 && distR <= 20) {
      delay(50);
      calculate_distance(trigger2, echo2);
      distR = dist;
      if (distR >= 10 && distR <= 20) {
         Serial.println("Right Locked");
         while (distR <= 40) {
            calculate_distance(trigger2, echo2);
            distR = dist;
            if (distR < 15) {
               Serial.println("Rewind");
               delay(300);
            }
            if (distR > 20) {
               Serial.println("Forward");
               delay(300);
            }
         }
      }
   }
   if (distR <= 8 && distR >= 0) {
      temp = millis();
      while (millis() <= (temp + 300))
         find_distance();

      Serial.println("next");
   }

   delay(200);
}
```

These lines define the **digital pins** connected to two **ultrasonic sensors. Trigger pin to send pulse and Echo pin to receive reflected pulse.**

```
const int trigger1 = 5; // Trigger pin of 1st Sensor
const int echo1 = 6;    // Echo pin of 1st Sensor
const int trigger2 = 9; // Trigger pin of 2nd Sensor
const int echo2 = 10;   // Echo pin of 2nd Sensor
```

We then define some global variables

```
long time_taken;
int dist, distL, distR;

long duration;
float r;
unsigned long temp = 0;
int temp1 = 0;
int l = 0;
```

This function **measures distance from both sensors**. Sends a short pulse (10 µs) on each trigger pin. Waits for the echo and measures the **pulse width** using pulseIn().

$$\text{distance} = \frac{3.4 \times \text{duration}}{2 \times 100}$$

Stores the left distance in distL and right distance in distR.

```
void find_distance(void);
void calculate_distance(int trigger, int echo);

void find_distance(void) {
    digitalWrite(trigger1, LOW);
    delayMicroseconds(2);
    digitalWrite(trigger1, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigger1, LOW);
    duration = pulseIn(echo1, HIGH, 5000);
    r = 3.4 * duration / 2;
    distL = r / 100.00;

    digitalWrite(trigger2, LOW);
    delayMicroseconds(2);
    digitalWrite(trigger2, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigger2, LOW);
    duration = pulseIn(echo2, HIGH, 5000);
    r = 3.4 * duration / 2;
    distR = r / 100.00;
    delay(100);
}
```

Serial communication for debugging (Serial Monitor). Configures trigger pins as **output** and echo pins as **input**.

```
void setup() {
    Serial.begin(9600);
    pinMode(trigger1, OUTPUT);
    pinMode(echo1, INPUT);
    pinMode(trigger2, OUTPUT);
    pinMode(echo2, INPUT);
```

Calculates the distance and caps it max to 50 cm and stores it and print it in serial monitor.

```
void calculate_distance(int trigger, int echo) {
    digitalWrite(trigger, LOW);
    delayMicroseconds(2);
    digitalWrite(trigger, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigger, LOW);
    time_taken = pulseIn(echo, HIGH);
    dist = time_taken * 0.034 / 2;
    if (dist > 50)
        dist = 50;
}

void loop() { // infinite loop
    calculate_distance(trigger1, echo1);
    distL = dist; // get distance of left sensor

    calculate_distance(trigger2, echo2);
    distR = dist; // get distance of right sensor

    // Uncomment for debugging
    Serial.print("L=");
    Serial.println(distL);
    Serial.print("R=");
    Serial.println(distR);
```

Video play pause mode

```
// Pause Modes - Hold
if ((distL > 25 && distR > 25) && (distL < 50 && distR < 50)) { //
    Serial.println("Play/Pause");
    delay(500);
}

calculate_distance(trigger1, echo1);
distL = dist;

calculate_distance(trigger2, echo2);
distR = dist;
```

This will control the volume when we move hand close volume increase and when we move hand away volume decreases.

```
if (distL >= 10 && distL <= 20) {
    delay(50); // Hand Hold Time
    calculate_distance(trigger1, echo1);
    distL = dist;
    if (distL >= 10 && distL <= 20) {
        Serial.println("Left Locked");
        while (distL <= 40) {
            calculate_distance(trigger1, echo1);
            distL = dist;
            if (distL < 15) { // Hand pushed in
                Serial.println("Vup");
                delay(300);
            }
            if (distL > 20) { // Hand pulled out
                Serial.println("Vdown");
                delay(300);
            }
        }
    }
}
```

It controls the forward and rewind function of the media player.

```
// Lock Right - Control Mode
if (distR >= 10 && distR <= 20) {
    delay(50); // Hand Hold Time
    calculate_distance(trigger2, echo2);
    distR = dist;
    if (distR >= 10 && distR <= 20) {
        Serial.println("Right Locked");
        while (distR <= 40) {
            calculate_distance(trigger2, echo2);
            distR = dist;
            if (distR < 15) { // Right hand pushed in
                Serial.println("Rewind");
                delay(300);
            }
            if (distR > 20) { // Right hand pulled out
                Serial.println("Forward");
                delay(300);
```

If right hand comes **very close (≤8 cm)**, interpreted as **Swipe Next** gesture and them that is the final delay produced of 200 ms.

```
// Swipe Next - Control Mode
if (distR <= 8 && distR >= 0) {
    temp = millis();
    while (millis() <= (temp + 300))
        find_distance();

    Serial.println("next");
}

delay(200);
```

Code: (Python code)

The python program for this project is very simple. We just have to establish a serial communication with Arduino through the correct baud rate and then perform some basic keyboard actions. The first step with python would be to install the **pyautogui** module. Make sure you follow this step because the program will not work without **pyautogui module.**

```
import serial  # Serial imported for Serial communication
import time    # Required to use delay functions
import pyautogui

ArduinoSerial = serial.Serial('com3', 9600)  # Create Serial port object called arduinoSerialData
time.sleep(2)  # Wait for 2 seconds for the communication to get established

while True:
    incoming = str(ArduinoSerial.readline().decode())  # Read the serial data and decode from
bytes
    print(incoming)  # Fixed print for Python 3

       if 'Play/Pause' in incoming:

        pyautogui.typewrite(['space'], 0.2)

    if 'Rewind' in incoming:
        pyautogui.hotkey('ctrl', 'left')

    if 'Forward' in incoming:
        pyautogui.hotkey('ctrl', 'right')

    if 'Vup' in incoming:
        pyautogui.hotkey('ctrl', 'down')

    if 'Vdown' in incoming:
        pyautogui.hotkey('ctrl', 'up')

    if 'next' in incoming:
        pyautogui.hotkey('ctrl', 'x')

    incoming = ""
```

Installing pyautogui and pyserial modules for windows:

Download the Latest Python from official Website and then follow the below steps to install *pyautogui* for windows. If you are using other platforms the steps will also be more or less similar. Make sure your computer/Laptop is connected to internet and proceed with steps below

STEP 1: Open Windows Command prompt and change the directory to the folder where you have installed python. By default the command should be

```
cd C:\Python27
```

STEP 2: Inside your python directory use the command *python –m pip install –upgrade pip* to upgrade your pip. Pip is a tool in python which helps us to install python modules easily. Once this module is upgraded (as shown in picture below) proceed to next step.

```
python -m pip install --upgrade pip
```

STEP 3: Use the command "*python –m pip install pyautogui*" to install the pyautogui module. Once the process is successful you should see a screen something similar to this below.

```
python -m pip install pyautogui
```

Refer the below Screen shot for series of commands to be typed on command prompt.

NOTE – Repeat the same process to install pyserial, replace pyautogui by pyserial in this process.

C:\Users\sraja\AppData\Local\Programs\Python\Python312\python.exe C:\Users\sraja\AppData\Local\Programs\Python\Python312\gesture.control.py

Next, we establish connection with the Arduino through COM port. In my computer the Arduino is connected to COM 2. Use device manager to find to which COM port your Arduino is connected to and correct the following line accordingly. Inside the infinite while loop, we repeatedly listen to the COM port and compare the key words with any pre-defied works and make key board presses accordingly.

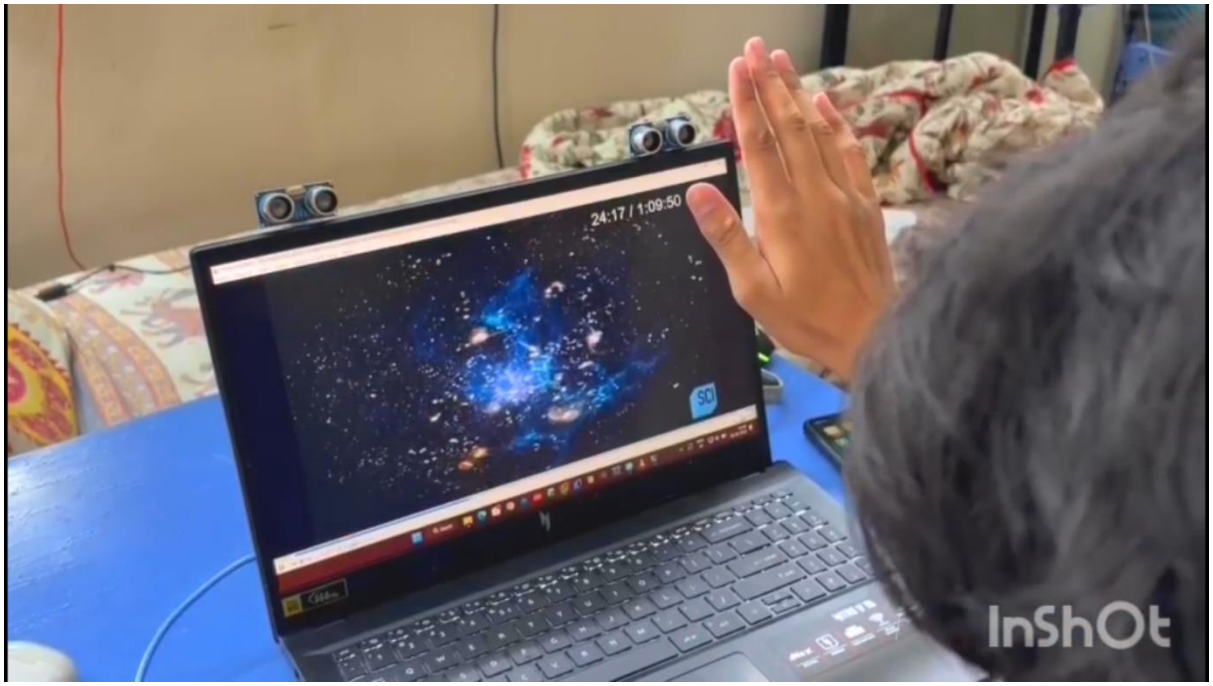
As you can see, to press a key we simply have to use the command "pyautogui.typewrite(['space'], 0.2)" which will press the key space for 0.2sec. If you need hot keys like ctrl+S then you can use the hot key command "pyautogui.hotkey('ctrl', 's')". I have used these combinations because they work on VLC media player you can tweak them in any way you like to create your own applications to control **anything in computer with gestures.**

Gesture controlled Computer in action

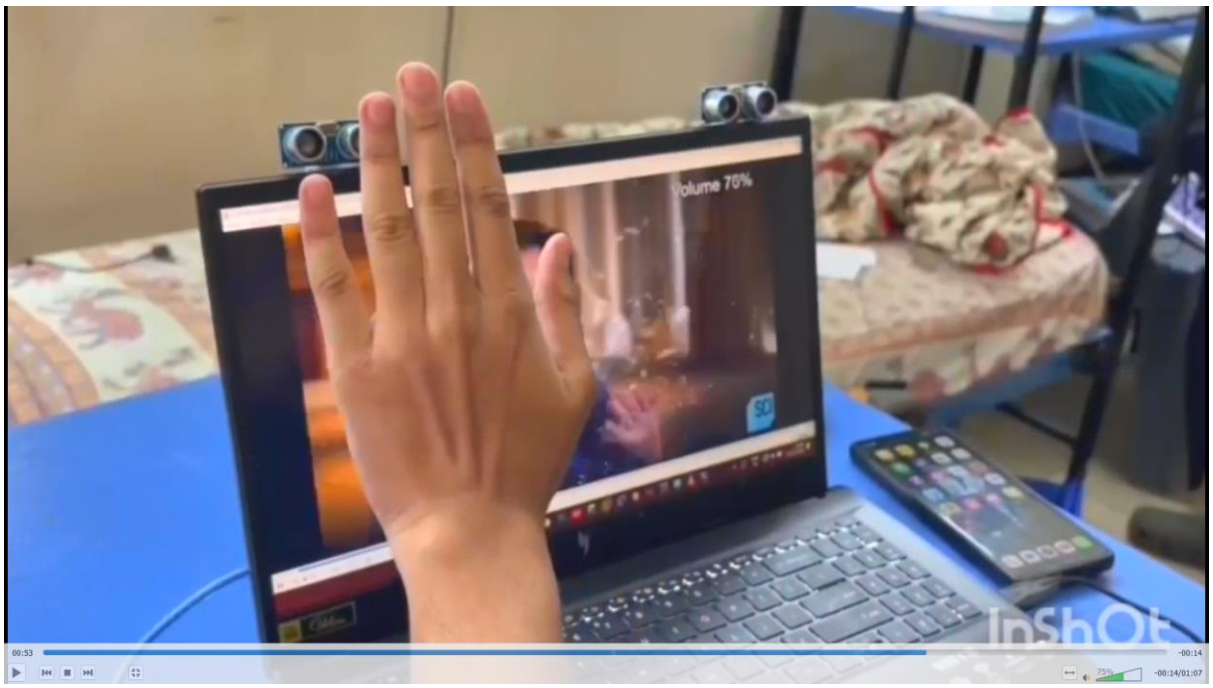
CIRCUIT:

PLAY/PAUSE:

FORWARD/REWIND:



VOLUME UP/DOWN:



https://drive.google.com/file/d/1MdnQXskgsOg6xsDrl0HZ7sqBsIohdeHR/view?usp=sharing

5. RESULTS & DISCUSSION

Gesture Recognition Accuracy:

The system achieved an accuracy upto 80 to 90% in correctly identifying all the gestures. This was observed when the user maintained a consistent hand movement pattern at a moderate speed and within the optimal distance range (3 to 50cm). The HC-SR04 ultrasonic sensor reliably captured distance variations, translating them into directional commands with minimal false positives.

Testing:

Extensive testing was conducted with hand gestures at varying distances.
Gesture Distance Range (cm) Detection Accuracy:

The system showed good responsiveness under normal lighting and stable hand movement. Noise and vibrations affected ultrasonic readings slightly, but averaging helped mitigate this.

6. FUTURE WORK

- Gesture recognition eases the level of handling a broad range of devices such as personal navigation devices, computers, laptops and mobile handsets.
- A growing trend towards touchless gesture recognition in niche applications including gaming is likely to amplify the growth of the touchless sensing market over the projected period.
- Additional gesture recognition opportunities exist in medical applications where, for health and safety reasons, a nurse or doctor may not be able to touch a display or track-pad but still needs to control a system.
- Appropriate gestures, such as hand swipes or using a finger as a virtual mouse, are a safer and faster way to control the device.

7. REFERENCES

1. Arduino Official Documentation. (n.d.). Arduino Reference Guide.Retrieved from https://www.arduino.cc/reference/en/
2. Processing Foundation. (n.d.). Processing Language Reference. Retrieved from https://processing.org/reference/
3. HC-SR04 Ultrasonic Sensor Datasheet. (n.d.). Retrieved from https://components101.com/
4. OpenCV Documentation. (n.d.). Open Source Computer Vision Library.Retrieved from https://docs.opencv.org/
5. Hand Gesture Gaming using Ultrasonic Sensors Arduinohttps://tinyurl.com/y684zwnp